
MSML 605

Concurrency

Thanks to Manoj Franklin for the slides' content

Code examples modified by Mohammad Nayeem Teli

Outline

- *Introduction*
 - *os.fork Function*
 - *os.system Function and os.exec Family of Functions*
 - *Controlling Process Input and Output*
 - *Interprocess Communication*
 - *Signal Handling*
 - *Sending Signals*
-

Introduction

- Implement concurrency by having each task (or process) operate in a separate memory space
 - Time slicing on single-processor systems divides processor time among many processes
 - OS allocates a short execution time called a quantum to a process
 - OS performs context switching to move processes and their dependent data in and out of memory
-

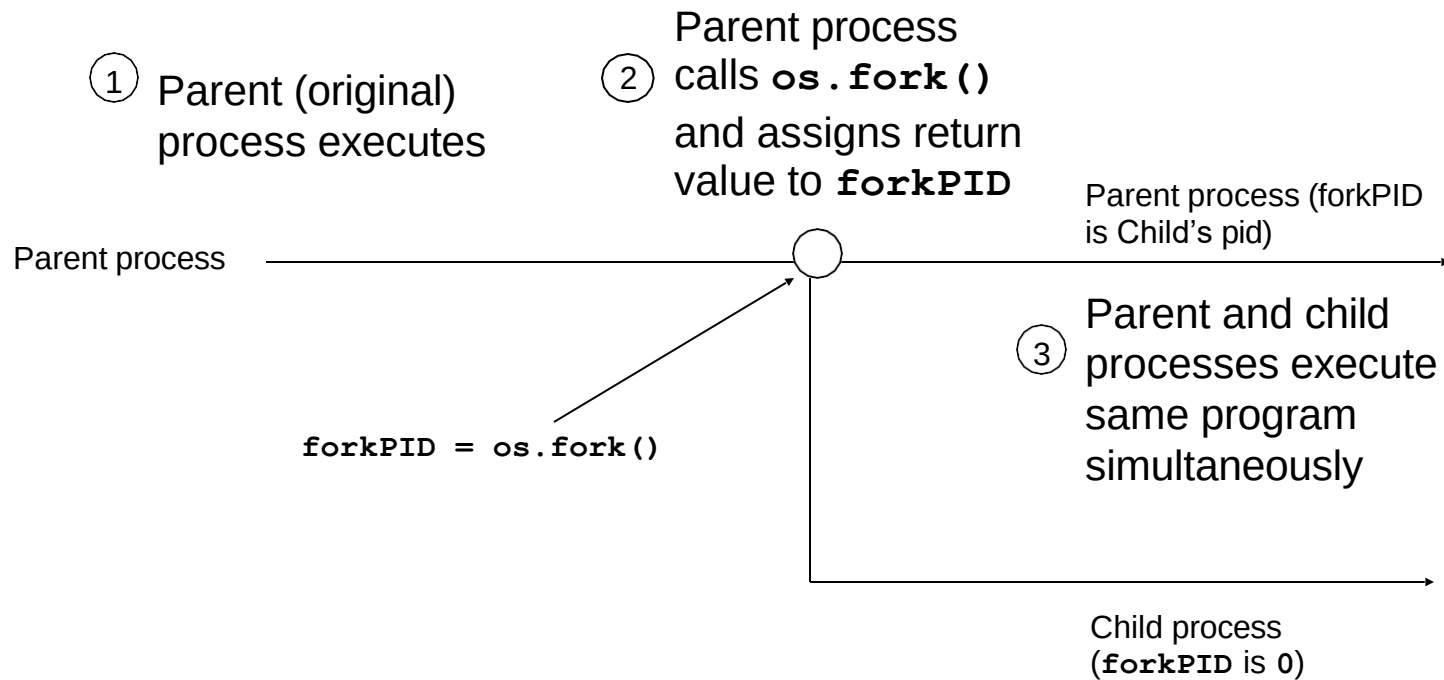
Introduction

- OS offers shells – which are software programs that carry out system commands for users.
 - Certain operating systems come equipped with system commands that allow programmers to develop and control processes.
 - For instance, the POSIX (Portable Operating System Interface for UNIX) standard outlines these system commands specifically for UNIX-based operating systems.
-

os Module

- **os** module provides functions for interacting with the operating system (OS)
 - Provides a portable way of using OS dependent functionality
 - Every function within the **os.fork** module will raise an **OSError** if provided with invalid or inaccessible file names, paths, or other correct but unacceptable arguments.
-

os.fork() Function



`os.fork()` creates a new process

```

1  # pid.py
2  # Using fork to create child processes.
3
4  import os
5  import sys
6  import time
7
8  processName = "parent" # only the parent is running now
9
10 print ("Program executing\n\tpid: %d, processName: %s" \
11        % ( os.getpid(), processName ))
12
13 # attempt to fork child process
14 try:
15     forkPID = os.fork() # create child process
16 except OSError:
17     sys.exit("Unable to create new process." )
18
19 if forkPID != 0: # am I parent process?
20     print ("Parent executing\n\t" + \
21           "\tpid: %d, forkPID: %d, processName: %s" \
22           % ( os.getpid(), forkPID, processName ))
23
24 elif forkPID == 0: # am I child process?
25     processName = "child"
26     print ("Child executing\n\t" + \
27           "\tpid: %d, forkPID: %d, processName: %s" \
28           % ( os.getpid(), forkPID, processName ))
29
30 print ("Process finishing\n\tpid: %d, processName: %s" \
31        % ( os.getpid(), processName ))

```

Creates duplicate of current process

os.fork raises **OSError** exception if fork failed

Parent process

Child process has *pid* of 0

```
Program executing
  pid: 6537, processName: parent
Parent executing
  pid: 6537, forkPID: 6538, processName: parent
Process finishing
  pid: 6537, processName: parent
Child executing
  pid: 6538, forkPID: 0, processName: child
Process finishing
  pid: 6538, processName: child
```

```
Program executing
  pid: 6664, processName: parent
Parent executing
  pid: 6664, forkPID: 6665, processName: parent
Process finishing
  pid: 6664, processName: parent
Child executing
  pid: 6665, forkPID: 0, processName: child
Process finishing
  pid: 6665, processName: child
```



```
# wait.py
# Demonstrates the wait function.
```

```
import os
import sys
import time
import random
```

```
# generate random sleep times for child processes
sleepTime1 = random.randrange( 1, 6 )
sleepTime2 = random.randrange( 1, 6 )
```

```
# parent ready to fork first child process
```

```
try:
    forkPID1 = os.fork()  # create first child process
except OSError:
    sys.exit( "Unable to create first child. " )
```

```
if forkPID1 != 0:    # am I parent process?
```

```
# parent ready to fork second child process
```

```
try:
    forkPID2 = os.fork()    # create second child process
except OSError:
    sys.exit( "Unable to create second child." )
```

```
if forkPID2 != 0: # am I parent process?
    print ("Parent waiting for child processes...\n" + \
          "\tupid: %d, forkPID1: %d, forkPID2: %d" \
          % ( os.getpid(), forkPID1, forkPID2 ))
```

Function **os.wait** causes parent to wait for child process to complete execution

```
# wait for any child process
try:
    child1 = os.wait()[ 0 ] # wait returns one child's pid
except OSError:
    sys.exit( "No more child processes." )
```

```
print ( "Parent: Child %d finished first, one child left." % child1)
```

```
# wait for another child process
try:
    child2 = os.wait()[ 0 ] # wait returns other child's pid
except OSError:
    sys.exit( "No more child processes." )
```

Function **os.wait** returns finished child's *pid*

```
print ( "Parent: Child %d finished second, no children left." % child2)
```

```
elif forkPID2 == 0: # am I second child process?
    print ( """Child2 sleeping for %d seconds...
            \tpid: %d, forkPID1: %d, forkPID2: %d""" \
            % ( sleepTime2, os.getpid(), forkPID1, forkPID2 ))
    time.sleep( sleepTime2 ) # sleep to simulate some work
```

```
elif forkPID1 == 0: # am I first child process?
    print ( """Child1 sleeping for %d seconds...
            \tpid: %d, forkPID1: %d""" \
            % ( sleepTime1, os.getpid(), forkPID1 ))
    time.sleep( sleepTime1 ) # sleep to simulate some work
```

```
Parent waiting for child processes...
  pid: 6887, forkPID1: 6888, forkPID2: 6889
Child1 sleeping for 3 seconds...
  pid: 6888, forkPID1: 0
Child2 sleeping for 3 seconds...
  pid: 6889, forkPID1: 6888, forkPID2: 0
Parent: Child 6888 finished first, one child left.
Parent: Child 6889 finished second, no children left.
```

```
Parent waiting for child processes...
  pid: 6898, forkPID1: 6899, forkPID2: 6900
Child1 sleeping for 1 seconds...
  pid: 6899, forkPID1: 0
Child2 sleeping for 3 seconds...
  pid: 6900, forkPID1: 6899, forkPID2: 0
Parent: Child 6899 finished first, one child left.
Parent: Child 6900 finished second, no children left.
```

```
Parent waiting for child processes...
  pid: 6901, forkPID1: 6902, forkPID2: 6903
Child1 sleeping for 3 seconds...
  pid: 6902, forkPID1: 0
Child2 sleeping for 3 seconds...
  pid: 6903, forkPID1: 6902, forkPID2: 0
Parent: Child 6903 finished first, one child left.
Parent: Child 6902 finished second, no children left.
```

```
# waitpid.py
# Demonstrates the waitpid function.
```

```
import os
import sys
import time
```

```
# parent about to fork first child process
```

```
try:
```

```
    forkPID1 = os.fork() # create first child process
```

```
except OSError:
```

```
    sys.exit( "Unable to create first child. " )
```

```
if forkPID1 != 0: # am I parent process?
```

```
# parent about to fork second child process
```

```
try:
```

```
    forkPID2 = os.fork() # create second child process
```

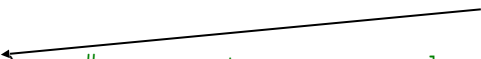
```
except OSError:
```

```
    sys.exit( "Unable to create second child." )
```

Create first child process



Create second child process



```
if forkPID2 > 0: # am I parent process?
    print "Parent waiting for child processes...\n" + \
        "\tupid: %d, forkPID1: %d, forkPID2: %d" \
        % ( os.getpid(), forkPID1, forkPID2 )

    # wait for second child process explicitly
    try:
        child2 = os.waitpid(forkPID2, 0 )[ 0 ] # child's pid
    except OSError:
        sys.exit("No child process with pid %d." % (forkPID2 ))

    print "Parent: Child %d finished." % child2

elif forkPID2 == 0: # am I second child process?
    print "Child2 sleeping for 4 seconds...\n" + \
        "\tupid: %d, forkPID1: %d, forkPID2: %d" \
        % ( os.getpid(), forkPID1, forkPID2 )
    time.sleep( 4 )

elif forkPID1 == 0: # am I first child process?
    print "Child1 sleeping for 2 seconds...\n" + \
        "\tupid: %d, forkPID1: %d" % ( os.getpid(), forkPID1 )
    time.sleep( 2 )
```

```
Parent waiting for child processes...  
    pid: 7084, forkPID1: 7089, forkPID2: 7090  
Child1 sleeping for 2 seconds...  
    pid: 7089, forkPID1: 0  
Child2 sleeping for 4 seconds...  
    pid: 7090, forkPID1: 7089, forkPID2: 0  
Parent: Child 7090 finished.
```

```
Parent waiting for child processes...  
    pid: 7094, forkPID1: 7095, forkPID2: 7096  
Child1 sleeping for 2 seconds...  
    pid: 7095, forkPID1: 0  
Child2 sleeping for 4 seconds...  
    pid: 7096, forkPID1: 7095, forkPID2: 0  
Parent: Child 7096 finished.
```

os.system() Function

- Function **os.system()** executes a system command using a shell and then returns control to the original process (after the command completes)
 - This method is implemented by calling the Standard C function **system()**
 - If command generates any output, it is sent to the interpreter standard output stream
-

os.system() Function

```
# systemfunc.py
# Uses the system function to clear the screen.
```

```
import os
import sys
```

Function **os.system** executes system-specific clear command

```
def printInstructions( clearCommand ):
    os.system( clearCommand ) # clear display
```

```
    print """Type the text that you wish to save in this file.
    Type clear on a blank line to delete the contents of the file.
    Type quit on a blank line when you are finished"""
```

Clear command depends on OS

```
# determine operating system
if os.name == "nt" or os.name == "dos": # Windows system
    clearCommand = "cls"
    print "You are using a Windows system."
elif os.name == "posix": # UNIX-compatible system
    clearCommand = "clear"
    print "You are using a UNIX-compatible system."
else:
    sys.exit( "Unsupported OS" )
```

```
filename = raw_input( "What file would you like to create? " )
```


os.system() Function

```
# open file
try:
    file = open( filename, "w+" )
except IOError, message:
    sys.exit( "Error creating file: %s" % message )

printInstructions( clearCommand )
currentLine = ""

# write input to file
while currentLine != "quit\n":
    file.write( currentLine )
    currentLine = sys.stdin.readline()

    if currentLine == "clear\n":

        # seek to beginning and truncate file
        file.seek( 0, 0 )
        file.truncate()

        currentLine = ""
        printInstructions( clearCommand )

file.close()
```

os.system() Function

```
You are using a UNIX-compatible system.  
What file would you like to create?
```

```
Type the text that you wish to save in this file.  
Type clear on a blank line to delete the contents of the file.  
Type quit on a blank line when you are finished.
```

os . exec Family of Functions

- **os . exec** family of functions do not return control to calling process after executing the specified command

os.exec Family of Functions

```
python webpage.py http://www.cs.umd.edu cs.txt
```

```
<!DOCTYPE html>
<html lang="en" dir="ltr" prefix="content: http://purl.org/rss/1.0/modules/content/ dc: http://purl.org/dc/terms/ foaf:
http://xmlns.com/foaf/0.1/ rdfs: http://www.w3.org/2000/01/rdf-schema# sioc: http://rdfs.org/sioc/ns# sioc: http://rdfs.org/sioc/types# skos: http://www.w3.org/2004/02/skos/core# xsd: http://www.w3.org/2001/XMLSchema#">
<head>
  <link rel="profile" href="http://www.w3.org/1999/xhtml/vocab" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
  <link rel="shortcut icon" href="https://www.cs.umd.edu/sites/all/themes/cs_gogo/favicon.ico" type="image/vnd.microsoft.icon" />
  <meta name="description" content="Homepage of the University of Maryland&#039;s Department of Computer Science" />
  <link rel="image_src" href="https://www.cs.umd.edu/sites/default/files/cs_logo.png" />
  <link rel="canonical" href="https://www.cs.umd.edu/home" />
  <link rel="shortlink" href="https://www.cs.umd.edu/home" />
  <meta property="og:type" content="website" />
  <meta property="og:url" content="https://www.cs.umd.edu/" />
  <meta property="og:title" content="UMD Department of Computer Science" />
  <title>Welcome | UMD Department of Computer Science</title>
  <link type="text/css" rel="stylesheet" href="https://www.cs.umd.edu/sites/default/files/css/
css_lQaZfjVpwP_oGNqdtWCSpJTlEMqXdMiU84ekLLxQnc4.css" media="all" />
  <link type="text/css" rel="stylesheet" href="https://www.cs.umd.edu/sites/default/files/css/
css_j7JZunwKIqwvzJFjFklgxRXtNohQFJhkDZ7rptQrYI.css" media="all" />
  <link type="text/css" rel="stylesheet" href="https://www.cs.umd.edu/sites/default/files/css/
css_aKOkUxCOQLcDgrXMyAhVibRlc6JIPg4m4_p5wbm6lHM.css" media="all" />
  <link type="text/css" rel="stylesheet" href="https://cdn.jsdelivrivr.net/npm/bootstrap@3.4.1/dist/css/bootstrap.min.css"
media="all" />
  <link type="text/css" rel="stylesheet" href="https://cdn.jsdelivrivr.net/npm/@unicorn-fail/drupal-bootstrap-styles@0.0.2/dist/
3.3.1/7.x-3.x/drupal-bootstrap.min.css" media="all" />
  <link type="text/css" rel="stylesheet" href="https://www.cs.umd.edu/sites/default/files/css/
css_1D7zGf7DYakhSXPzaG8NghJjfba3LavXmQ7ZjhLLuPo.css" media="all" />
  <!-- HTML5 element support for IE6-8 -->
```

os.exec Family of Functions

Function name	Description
exec1 (<i>path</i> , <i>arg0</i> , <i>arg1</i> , ...)	Executes the program <i>path</i> with the given arguments.
execle (<i>path</i> , <i>arg0</i> , <i>arg1</i> , ..., <i>environment</i>)	Same as exec1 , but takes an additional argument—the <i>environment</i> in which the program executes. The environment is a dictionary with keys that are environment variables and with values that are environment variable values.
exec1p (<i>path</i> , <i>arg0</i> , <i>arg1</i> , ...)	Same as exec1 , but searches for executable <i>path</i> in <code>os.environ["PATH"]</code> .
execv (<i>path</i> , <i>arguments</i>)	Same as exec1 , but arguments are contained in a single tuple or list.
excve (<i>path</i> , <i>arguments</i> , <i>environment</i>)	Same as execle , but arguments are contained in a single tuple or list.
execvp (<i>path</i> , <i>arguments</i>)	Same as exec1p , but arguments are contained in a single tuple or list.
execvpe (<i>path</i> , <i>arguments</i> , <i>environment</i>)	Same as execvp , but takes an additional argument, <i>environment</i> , in which the program executes.

Controlling Process Input /Output

- UNIX-system users can use multiple programs together to achieve a particular result (e.g., `ls | sort` produces a sorted list of a directory's contents)

Controlling Process Input /Output

```
#subpro.py
import os
import subprocess
```

Determine OS to set system-specific directory-listing and reverse-sort commands

```
# determine operating system, then set directory-listing and
reverse-sort commands
```

```
if os.name == "nt" or os.name == "dos": # Windows system
```

```
    fileList = "dir /B"
```

```
    sortReverse = "sort /R"
```

```
elif os.name == "posix": # UNIX-compatible system
```

```
    fileList = "ls -l"
```

```
    sortReverse = "sort -r"
```

```
else:
```

```
    sys.exit("OS not supported by this program.")
```

Executes command and obtains **stdout** stream

```
# obtain stdout of directory-listing command
```

```
dirOut = subprocess.Popen(fileList, shell=True,
stdout=subprocess.PIPE).stdout
```

Executes command and obtains **stdout** and **stdin** streams

```
# obtain stdout of reverse-sort command
```

```
sortOut = subprocess.Popen(sortReverse, shell=True,
stdin=subprocess.PIPE, stdout=subprocess.PIPE)
```

Returns directory-listing command's output

```
filenames = dirOut.read().decode() # output from directory-listing
command
```

Controlling Process Input /Output

```
# display output from directory-listing command
```

```
print("Before sending to sort")
```

```
print("(Output from '%s'):" % fileList)
```

```
print(filenamees)
```

```
# send output to stdin of sort command and get sorted output
```

```
sorted_output, _ = sortOut.communicate(input=filenamees.encode())
```

```
sortOut.stdout.close() # close stdout of sort command
```

```
# display sorted output
```

```
print("After sending to sort")
```

```
print("(Output from '%s'):" % sortReverse)
```

```
print(sorted_output.decode()) # output from sort command
```


Controlling Process Input /Output

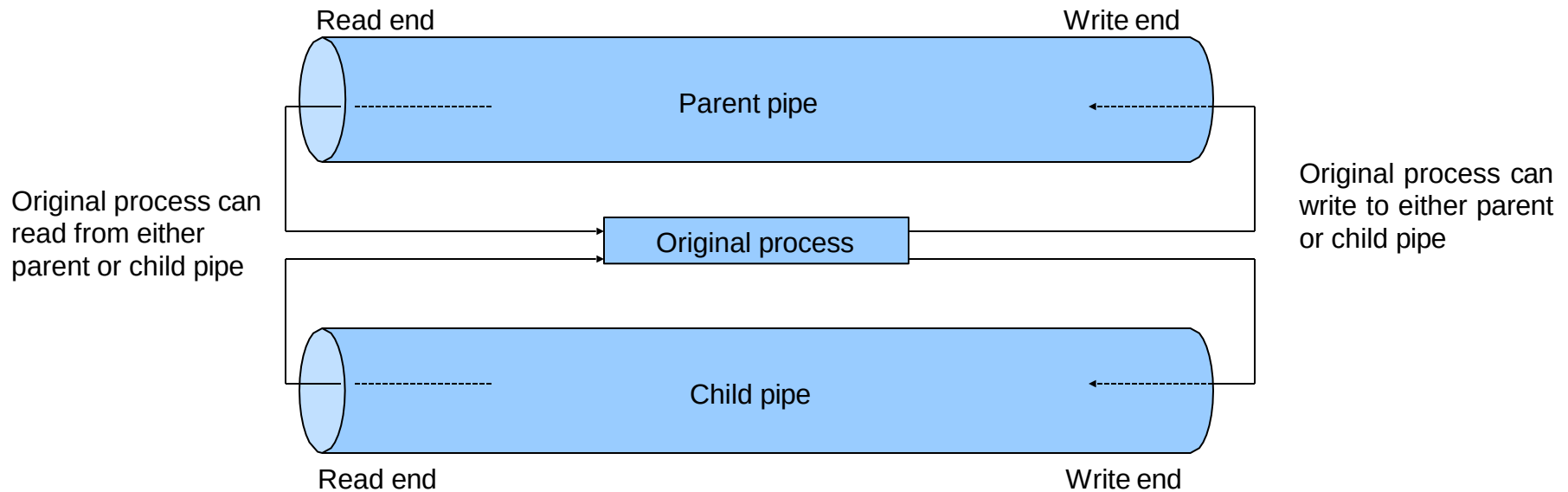
```
python subpro.py  
Before sending to sort  
(Output from 'ls -l'):  
pid.py  
subpro.py  
systemfunc.py  
wait.py  
waitpid.py  
waitpid1.py  
webpage.py
```

```
After sending to sort  
(Output from 'sort  
-r'):  
webpage.py  
waitpid1.py  
waitpid.py  
wait.py  
systemfunc.py  
subpro.py  
pid.py
```

Interprocess Communication

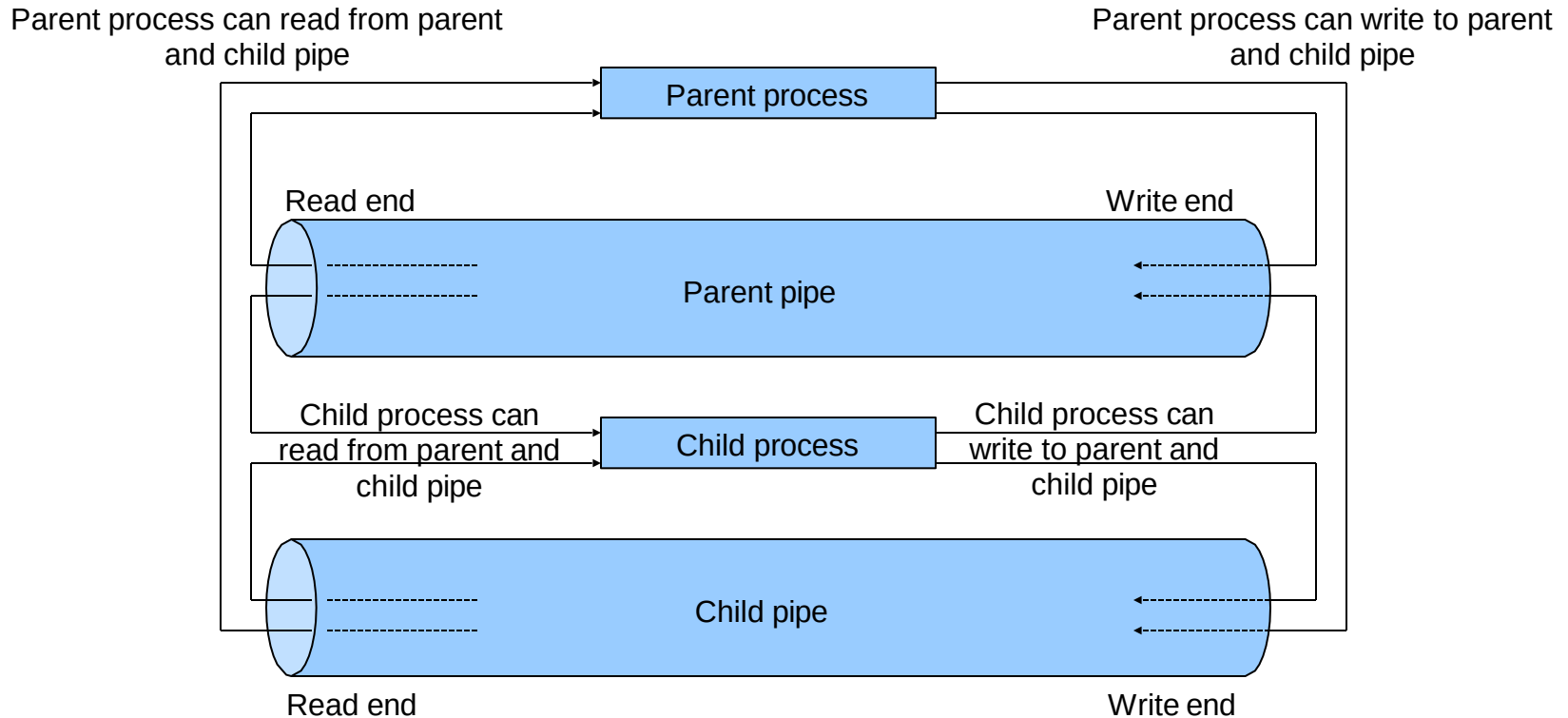
- Module **os** also provides interface to many interprocess communication (IPC) mechanisms
- Pipe, IPC mechanism, is a file-like object that provides a one-way communication channel
- Function **os.pipe()** creates a pipe and returns a tuple containing two file descriptors which provide read and write access to the pipe
- Functions **os.read()** and **os.write()** read and write the files associated with the file descriptors

Interprocess Communication



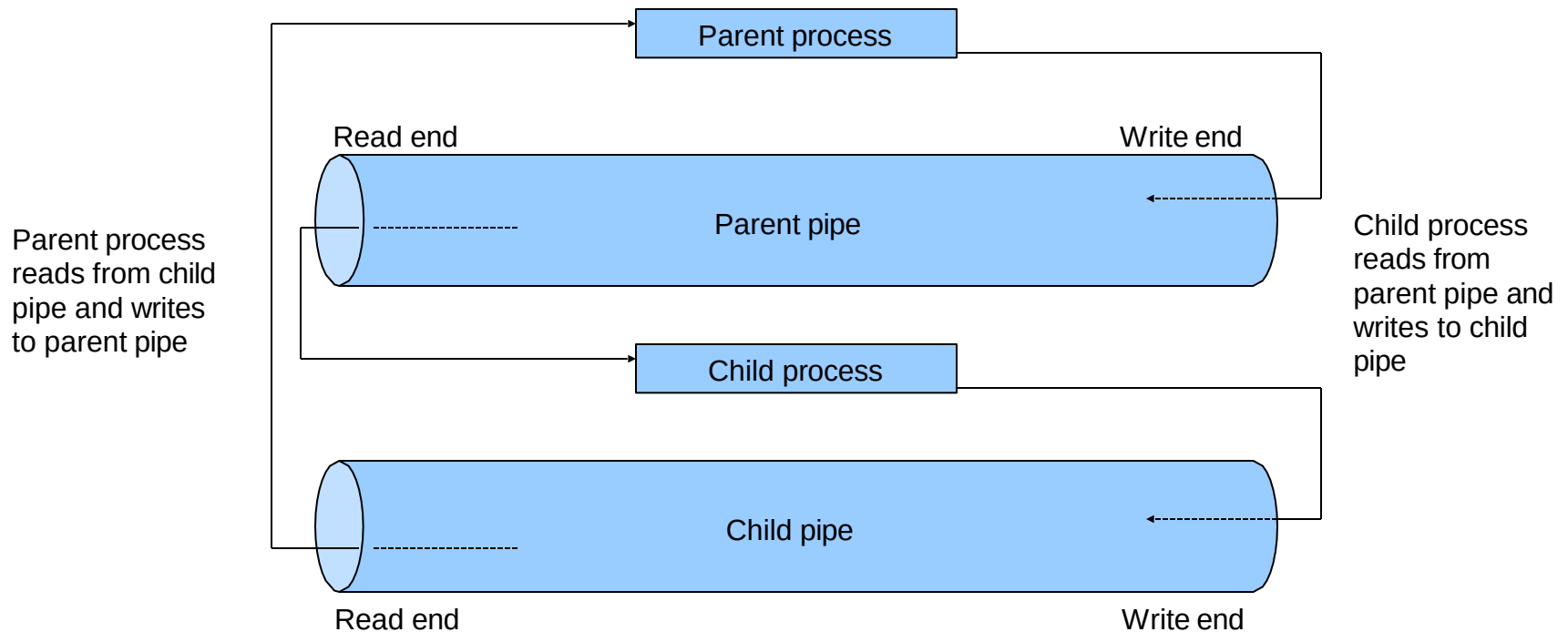
Initial stage: Original process can read and write to both pipes

Interprocess Communication



Intermediate stage: Parent and child processes can read and write to both pipes

Interprocess Communication



Final stage: Parent and child processes close unneeded ends of the pipes

```
# ospipe.py
# Using os.pipe to communicate with a child process.
3
4     import os
5     import sys
6
7     # open parent and child read/write pipes
8     fromParent, toChild = os.pipe()
9     fromChild, toParent = os.pipe()
10
11    # parent about to fork child process
12    try:
13        pid = os.fork()    # create child process
14    except OSError:
15        sys.exit( "Unable to create child process." )
16
17    if pid != 0:    # am I parent process?
18
19        # close unnecessary pipe ends
20        os.close( toParent )
21        os.close( fromParent )
22
23        # write values from 1-10 to parent's write pipe and
24        # read 10 values from child's read pipe
25        for i in range( 1, 11 ):
26            os.write( toChild, str( i ).encode() )
27            print(f"Parent sent: {i}")
28            response = os.read(fromChild, 64)    # Read response from child
29            print(f"Parent received: {response.decode()}")
```

```
31     else:  # Child process
32         # Close unnecessary pipe ends
33         os.close(toChild)
34         os.close(fromChild)
35         # Read values from parent and send modified
36         response back
37         for _ in range(10):
38             value = os.read(fromParent, 64).decode()  #
39             Read value from parent
40             modified_value = f"Child received: {value}"
41             # Modify the received value
42             os.write(toParent, modified_value.encode())
43             # Send modified value back to parent
```

```
Parent sent: 1
Parent received: Child received: 1
Parent sent: 2
Parent received: Child received: 2
Parent sent: 3
Parent received: Child received: 3
Parent sent: 4
Parent received: Child received: 4
Parent sent: 5
Parent received: Child received: 5
Parent sent: 6
Parent received: Child received: 6
Parent sent: 7
Parent received: Child received: 7
Parent sent: 8
Parent received: Child received: 8
Parent sent: 9
Parent received: Child received: 9
Parent sent: 10
Parent received: Child received: 10
```

Signal Handling

- Processes can communicate with signals, which are messages that the OS delivers to programs asynchronously
 - Signal processing: program receives a signal and performs an action based on that signal
 - Signal handlers execute in response to signals
 - Every Python program has default signal handlers
-

Signal Handling

```
#signalhandler.py
import time
import signal

def stop( signalNumber, frame ):
    global keepRunning
    keepRunning -= 1
    print ( "Ctrl-C pressed; keepRunning is", keepRunning)

keepRunning = 3

# set the handler for SIGINT to be function stop
signal.signal( signal.SIGINT, stop )

while keepRunning:
    print "Executing..."
    time.sleep( 1 )

print "Program terminating..."
```

```
Executing...
Executing...
Executing...
Executing...
Executing...
Executing...
Executing...
Executing...
Executing...
^C Ctrl-C pressed; keepRunning is 2
Executing...
Executing...
Executing...
Executing...
Executing...
Executing...
^C Ctrl-C pressed; keepRunning is 1
Executing...
Executing...
^C Ctrl-C pressed; keepRunning is 0
Program terminating...
```

Sending Signals

- Executing programs can send signals to other processes
- Function `os.kill()` sends a signal to a particular process identified by its *pid*

Sending Signals

```
1  # interprocesssignal.py
2  # Sending signals to child processes using kill
3
4  import os
5  import signal
6  import time
7  import sys
8
9  # handles both SIGALRM and SIGINT signals
10 def parentInterruptHandler( signum, frame ):
11     global pid
12     global parentKeepRunning
13
14     # send kill signal to child process and exit
15     os.kill( pid, signal.SIGKILL ) # send kill signal
16     print "Interrupt received. Child process killed."
17
18     # allow parent process to terminate normally
19     parentKeepRunning = 0
20
21 # set parent's handler for SIGINT
22 signal.signal( signal.SIGINT, parentInterruptHandler )
23
24 # keep parent running until child process is killed
25 parentKeepRunning = 1
```

Sending Signals

```
27 # parent ready to fork child process
28 try:
29     pid = os.fork() # create child process
30 except OSError:
31     sys.exit( "Unable to create child process." )
32
33 if pid != 0: # am I parent process?
34     while parentKeepRunning:
35         print "Parent running. Press Ctrl-C to terminate child."
36         time.sleep( 1 )
37
38 elif pid == 0: # am I child process?
39
40     # ignore interrupt in child process
41     signal.signal( signal.SIGINT, signal.SIG_IGN )
42
43     while 1:
44         print "Child still executing."
45         time.sleep( 1 )
46
47 print "Parent terminated child process."
48 print "Parent terminating normally."
```

```
Parent running. Press Ctrl-C to terminate child.  
Child still executing.  
Child still executing.  
Parent running. Press Ctrl-C to terminate child.  
Child still executing.  
Parent running. Press Ctrl-C to terminate child.  
Child still executing.  
Parent running. Press Ctrl-C to terminate child.  
Child still executing.  
Parent running. Press Ctrl-C to terminate child.  
Child still executing.  
Parent running. Press Ctrl-C to terminate child.  
Child still executing.  
Parent running. Press Ctrl-C to terminate child.  
Child still executing.  
Parent running. Press Ctrl-C to terminate child.  
Child still executing.  
Parent running. Press Ctrl-C to terminate child.  
Child still executing.  
Parent running. Press Ctrl-C to terminate child.  
Child still executing.  
^CInterrupt received. Child process killed.  
Parent terminated child process.  
Parent terminating normally.
```

Multithreading in Python: The Hidden Truth

Outline

Introduction

Thread States: Life Cycle of a Thread

threading.Thread Example

Thread Synchronization

Producer/Consumer without Synchronization

Producer/Consumer with Synchronization

Producer/Consumer: Module Queue

Producer/Consumer: The Circular Buffer

Semaphores

Events

Process

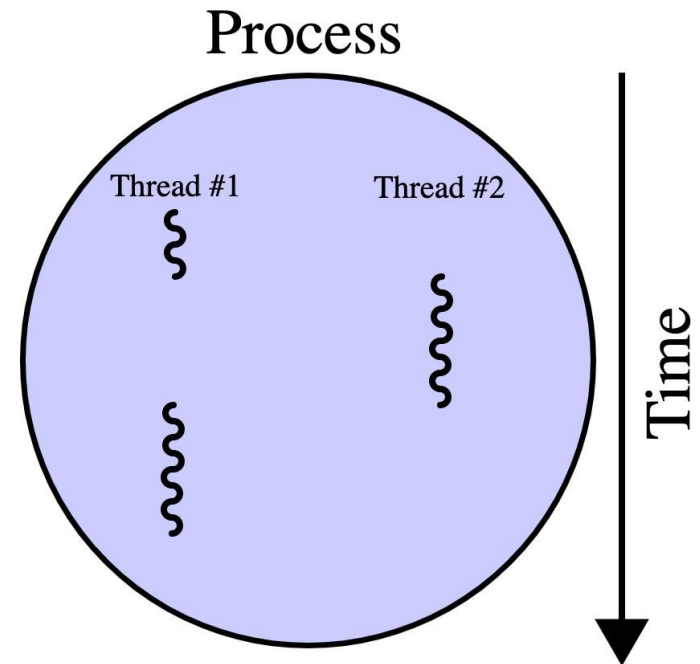
- *A unit of work, for example, Jupyter notebook*
 - *An OS can run multiple processes at the same time.*
 - *By default Python interpreter executes instructions serially.*
 - *The size of the datasets has increased.*
 - *The algorithms are more complex and need to process more, hence the need for multi-processing*
-

Parallel processing

- *It can be achieved in two ways: Multiprocessing and Threading*
 - *Process: An instance of a program
Uses its own memory space*
 - *Threads: components of a process, which can run in parallel*
 - *Multiple threads*
 - *Share parent process memory space*
-

Processes and Threads

- *Threads live in the same memory space*
- *Processes have their separate memory space*
- *Spawning processes is slower than spawning threads.*
- *Sharing objects between threads is easier.*
- *Inter-process communication between processes.*



[Source](#)

Processes and Threads

- A process is a program in execution
- A thread is one line of execution within a process. A process may contain many threads
- Thread creation has much less overhead than process creation, especially in Windows
- Each thread has its own stack (local variables), but share global variables with the other threads
- Global variables allow threads to share information and communicate with one another
- Shared data introduces the need for synchronization – A can of worms

Introduction

- Threads often called “light-weight” process because operating systems generally require fewer resources to create and manage threads than to create and manage processes
- Python’s threading capabilities **depend on whether the operating system supports multithreading**
- Python’s **threading** module provides its multithreading capabilities

Thread States: Life Cycle of a Thread

- Python interpreter controls all program threads
 - Python's global interpreter lock (GIL) ensures that the interpreter runs only one thread at any given time
 - Thread is said to be in one of several thread states at any time
 - Python programs can define threads by inheriting from class **threading.Thread** and overriding its functionality
-

THREAD STATES

Thread States

- Newly created thread begins its lifecycle in the **born** state
- Invoking the thread's **start** method places the thread in the **ready** state
- The thread's **run** method, which implements the tasks that the thread performs, obtains the GIL and enters the **running** state
- Thread enters the **dead** state when its **run** method returns or terminates for any reason (e.g., an uncaught exception)

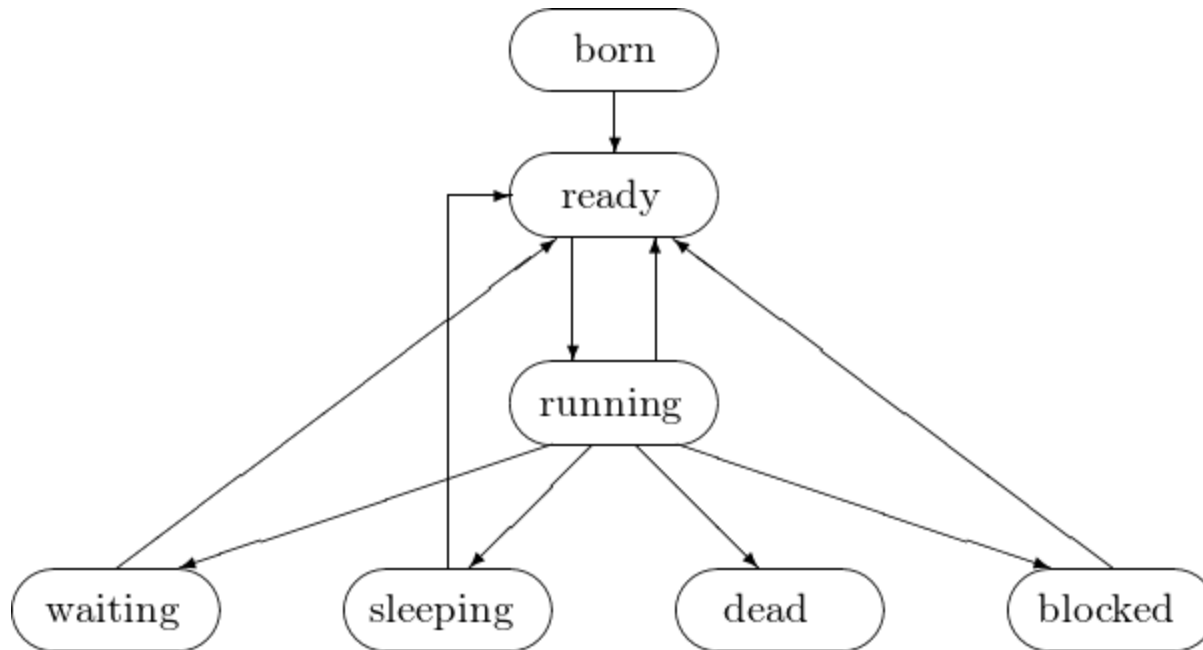
Thread States

- A running thread that calls another thread's **join** method forfeits the GIL and waits for the **joined** thread to die before proceeding
- Thread enters the blocked state while it waits for a requested, unavailable resource (e.g. I/O device)
- When a running thread calls function **time.sleep()**, it releases the GIL and enters the sleeping state

Thread States

- Deadlock occurs when one or more threads wait forever for an event that cannot occur
- In indefinite postponement, one or more threads is delayed for some unpredictably long time

Thread States



State diagram showing the life cycle of a thread

Thread Information

- Module **threading** provides ways for a program to obtain information about its threads, including their current states
 - Function **threading.currentThread()** returns reference to currently running **Thread** object
 - Function **threading.enumerate()** returns list of currently active **Thread** objects
 - Function **threading.activeCount()** returns length of list returned by **threading.enumerate()**
 - Method **isAlive()** returns 1 if the **Thread** object's **start** method has been invoked and the **Thread** object is not dead
 - Methods **setName()** and **getName()** allow programmer to set and get a **Thread** object's name, respectively

threading.Thread example

- Threads can be created by defining a class that derives from **threading.Thread** and instantiating objects of that class

Threads

```
# threads.py
# Multiple threads printing at different intervals.
3
4 import threading
5 import random
6 import time
7
8 class PrintThread( threading.Thread ):
9     """Subclass of threading.Thread"""
10
11     def __init__( self, threadName ):
12         """Initialize thread, set sleep time, print data"""
13
14         threading.Thread.__init__( self, name = threadName )
15         self.sleepTime = random.randrange( 1, 6 )
16         print "Name: %s; sleep: %d" % \
17             ( self.getName(), self.sleepTime )
18
19     # overridden Thread run method
20     def run( self ):
21         """Sleep for 1-5 seconds"""
22
23         print "%s going to sleep for %s second(s)" % \
24             ( self.getName(), self.sleepTime )
25         time.sleep( self.sleepTime )
26         print self.getName(), "done sleeping"
27
28 thread1 = PrintThread( "thread1" )
29 thread2 = PrintThread( "thread2" )
30 thread3 = PrintThread( "thread3" )
31
32 print "\nStarting threads"
33 # Start the threads
34 thread1.start()
35 thread2.start()
36 thread3.start()
```

Threads

```
Name: thread1; sleep: 3
Name: thread2; sleep: 4
Name: thread3; sleep: 1

Starting threads
thread1 going to sleep for 3 second(s)
thread2 going to sleep for 4 second(s)
thread3 going to sleep for 1 second(s)
thread3 done sleeping
thread1 done sleeping
thread2 done sleeping
```

THREAD SYNCHRONIZATION
